

Floricultura Rosa de Saron — catálogo e painel administrativo

Vitrine digital com carrinho no navegador e pedidos finalizados pelo WhatsApp.

STATUS: EM DESENVOLVIMENTO

Case técnico de Wilian Gulini — desenvolvedor front-end (React, Next.js, TypeScript, Angular).

Resumo executivo

A Floricultura Rosa de Saron é uma vitrine digital em desenvolvimento para uma floricultura local. O visitante navega por um catálogo de produtos e categorias, monta um pedido em um carrinho que vive no próprio navegador e envia esse pedido, já organizado, para o WhatsApp da loja. A confirmação de disponibilidade, prazo e pagamento acontece na conversa — o site não processa pagamento. Um painel administrativo protegido por autenticação própria permite gerenciar produtos, categorias, imagens e configurações da loja.

Status atual: em desenvolvimento

O projeto está em desenvolvimento ativo. Funcionalidades, conteúdo e decisões de interface ainda podem mudar. Este case descreve o que está implementado e comprovado no código até a data de revisão indicada na matriz de evidências do repositório, não um produto finalizado.

Contexto do negócio

Floriculturas pequenas costumam vender por WhatsApp, Instagram ou balcão, sem uma vitrine própria que organize o catálogo e facilite a decisão do cliente. A Floricultura Rosa de Saron precisava de um site que funcionasse como catálogo permanente — com fotos, descrições e preços claros — sem abrir mão do canal de atendimento que a loja já usa no dia a dia: o WhatsApp.

Problema que a aplicação busca resolver

Sem um catálogo digital, o cliente depende de perguntar item por item pelo WhatsApp para saber o que está disponível e por quanto. Isso gera atrito para o cliente e trabalho repetitivo para a loja. A aplicação move a etapa de "explorar o catálogo e montar o pedido" para o site, e mantém a etapa de "confirmar e negociar detalhes" no canal humano — o WhatsApp — onde a floricultura já tem controle total sobre preço, prazo e disponibilidade real.

Objetivo do produto

Oferecer uma vitrine simples, rápida em celular e organizada por SEO local, capaz de:

- apresentar produtos e categorias de forma clara;

- deixar o cliente montar um pedido sem precisar criar conta;
- transformar esse pedido em uma mensagem de WhatsApp pronta, legível e completa;
- dar à floricultura um painel simples para manter o catálogo atualizado sozinha.

Público e jornada principal

Público: pessoas buscando flores, buquês, arranjos ou presentes de uma floricultura local, majoritariamente pelo celular.

Jornada principal:

1. Chega pela busca local, redes sociais ou link direto.
2. Navega pelo catálogo ou por uma categoria específica.
3. Abre um produto para ver descrição, preço e disponibilidade.
4. Adiciona ao carrinho (sem login).
5. Revisa o carrinho: itens, quantidades, subtotais e total estimado.
6. Preenche a finalização do pedido: nome, modalidade (retirada ou entrega), data desejada, forma de pagamento, mensagem de cartão e observações.
7. Envia o pedido — o navegador abre o WhatsApp da loja com a mensagem pronta.
8. A floricultura confirma tudo diretamente na conversa.

Funcionalidades comprovadas

Área pública

- Página inicial com identidade da loja e destaques do catálogo.
- Catálogo de produtos (`/produtos`) e navegação por categoria (`/categoria/[slug]`).
- Página de detalhe do produto (`/produto/[slug]`) com tipos de preço distintos.
- Carrinho local (`/carrinho`) com quantidades, subtotais e total estimado.
- Finalização do pedido (`/pedido`) com validação inline e persistência de rascunho.
- Envio do pedido pelo WhatsApp com mensagem formatada e codificada.
- Páginas institucionais: Sobre e Contato.
- SEO local: metadados dinâmicos, Open Graph, Twitter Card, dados estruturados (`LocalBusiness` , `Product` , `BreadcrumbList`), `sitemap.xml` e `robots.txt` .

Área administrativa

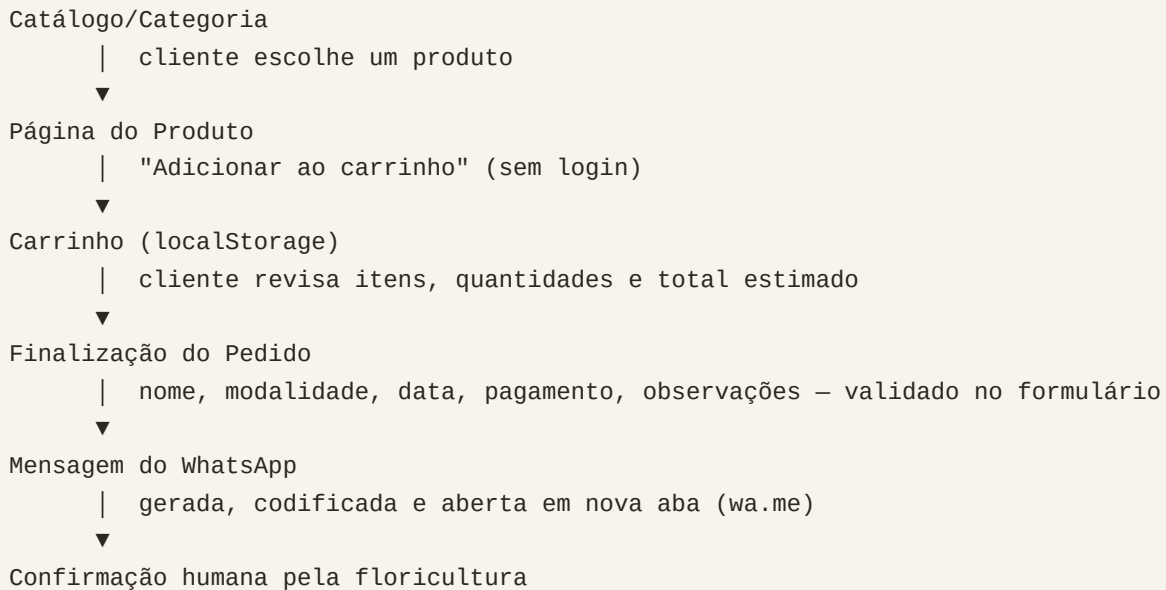
- Login com e-mail e senha, sessão protegida em todas as rotas `/admin/*` .
- Recuperação de senha por e-mail com token de uso único e expiração curta.
- Gerenciamento de produtos e categorias, incluindo upload de imagens.
- Confirmação explícita antes de qualquer exclusão.

- Gerenciamento de configurações gerais da loja (endereço, WhatsApp, redes sociais, horário) e do perfil da administradora.

Não implementado, por decisão de escopo

Pagamento online, conta de cliente, cupons, chat interno e integrações com ERP ou marketplaces. O projeto foi definido deliberadamente como vitrine + carrinho local + painel simples + pedido via WhatsApp.

Fluxo: catálogo → produto → carrinho → pedido → WhatsApp



Cada etapa é responsabilidade de uma camada isolada e testada: o carrinho é um `Context` de React com persistência em `localStorage`; a montagem da mensagem é uma função pura testada isoladamente; o envio é um link `https://wa.me` aberto em nova aba.

Painel administrativo

O painel foi desenhado para ser operado por uma pessoa sem conhecimento técnico:

- uma ação por tela, sem menus escondidos;
- todo texto, rótulo e mensagem de erro em português, sem jargão;
- toda ação de excluir passa por um diálogo de confirmação explícito;
- toda ação de salvar, editar ou excluir dá feedback visual imediato de sucesso ou erro;
- a navegação é a mesma em desktop e celular, adaptada por layout responsivo.

O acesso é feito em `/admin/login`; toda rota abaixo de `/admin/*` exige sessão válida, verificada em um middleware dedicado a cada requisição.

Atuação de Wilian Gulini

Wilian conduziu o desenvolvimento do projeto: definição da arquitetura de pastas, modelagem de dados, implementação da interface pública e do painel administrativo, construção do fluxo de carrinho e checkout, integração com WhatsApp, autenticação administrativa, upload de imagens e escrita da suíte de testes. O projeto usa apoio de ferramentas de IA no processo — ver "Como este projeto foi construído" — sempre sob revisão humana e regras de projeto versionadas no próprio repositório.

Stack principal

- **Next.js** com App Router
- **React**
- **TypeScript** em modo estrito
- **Tailwind CSS**

Front-end é o eixo deste case: estrutura de rotas do App Router, Server e Client Components, formulários controlados com validação inline, contexto de carrinho, acessibilidade (`aria-*` , foco gerenciado em modais) e um design system próprio construído sobre tokens do Tailwind.

Capacidades complementares aplicadas

Aplicadas ao projeto como suporte ao front-end, não como foco principal do case:

- **Prisma + PostgreSQL** para persistência de produtos, categorias, imagens, configurações e usuários administrativos.
- **Autenticação própria** (HMAC-SHA256, cookie assinado, hash de senha) para proteger o painel, sem depender de serviço de terceiros.
- **Cloudinary** para upload e validação de imagens.
- **Vitest** para testes unitários e **Playwright** para testes end-to-end.
- Um script de deploy próprio para publicar a aplicação em ambiente próprio.

Decisões de interface e responsividade

- Design system próprio ("Ateliê Botânico"): paleta de marca e tokens semânticos (`background` , `surface` , `foreground` , `primary`) definidos uma vez em `globals.css` e reaproveitados por todos os componentes de interface.
- Componentes de UI reutilizáveis (botão, cartão, campo, modal, toast, estado vazio) compartilhados entre a área pública e o painel administrativo.
- Alvo de toque mínimo de 44px (`min-h-11`) em botões e controles interativos.
- Menu público adaptado: navegação horizontal em telas largas, menu hambúrguer acessível (fecha com `Escape` , devolve foco ao botão) em telas estreitas.

- Modais de confirmação usam `role="alertdialog"`, bloqueiam fechamento durante o envio e devolvem o foco ao elemento que os abriu.

Decisões técnicas relevantes

O carrinho trata o próprio armazenamento como fonte não confiável. Os itens vivem em `localStorage`, o que dispensa cadastro do cliente, mas significa que qualquer coisa pode estar lá na próxima visita. A leitura valida cada campo antes de aceitar o item: descarta registros malformados, normaliza a quantidade para um inteiro positivo, zera o preço de itens sob consulta, deduplica por produto somando quantidades e limpa a chave inteira quando o JSON não faz sentido. Um carrinho corrompido resulta em carrinho vazio, nunca em tela quebrada ou total incorreto.

Três tipos de preço, uma regra de total. Um produto pode ter preço fixo, preço "a partir de" ou valor sob consulta. Os dois primeiros entram no total estimado exibido ao cliente; o terceiro nunca entra, e a mensagem final avisa isso explicitamente. Essa regra está isolada em funções puras e é a parte mais testada do projeto.

Autenticação sem framework externo. A sessão administrativa usa um cookie assinado com HMAC-SHA256 via Web Crypto API, com janela deslizante de renovação e comparação de assinatura resistente a timing attack. A decisão prioriza uma superfície de dependência menor para um painel de uso simples, em vez de introduzir uma biblioteca de autenticação completa para um único papel de usuário.

Separação clara entre área pública e área administrativa. Rotas, componentes e lógica de servidor da área `/admin` não se misturam com a área pública; um middleware único intercepta toda rota administrativa e decide acesso antes de qualquer página renderizar.

Qualidade e testes

- Testes unitários com Vitest cobrindo, entre outros pontos, a montagem da mensagem do WhatsApp, o cálculo de totais, a normalização de dados do carrinho, a geração de slugs, a formatação de telefone e moeda, e as ações de servidor do painel administrativo.
- Testes end-to-end com Playwright cobrindo autenticação, recuperação de senha, páginas públicas e administrativas, e responsividade em múltiplos viewports (incluindo verificação de ausência de rolagem horizontal e comportamento do menu por teclado). A suíte existe e está versionada no repositório; este case não declara execução recente dela.
- Lint (ESLint) e checagem de tipos (TypeScript em modo estrito) fazem parte do fluxo de validação do projeto.

Segurança e privacidade

Princípios aplicados, sem detalhar mecanismos exploráveis:

- Segredos e credenciais ficam exclusivamente em variáveis de ambiente, nunca no código ou no front-end.
- Senhas de administrador são armazenadas com hash, nunca em texto plano.

- Tokens de recuperação de senha são de uso único, expiram em pouco tempo e o banco armazena apenas um hash do token — nunca o valor enviado por e-mail.
- Dados vindos do carrinho do cliente são tratados como não confiáveis e validados antes de qualquer uso.
- Upload de imagem é validado no servidor pelo conteúdo real do arquivo, não apenas pela extensão, com formatos e tamanho limitados.
- Toda rota administrativa passa por verificação de sessão antes de qualquer resposta.

Limitações atuais

- Projeto em desenvolvimento: funcionalidades e conteúdo ainda mudam.
- Sem pagamento online e sem conta de cliente — decisão de escopo, não uma pendência.
- O carrinho vive no navegador: não sincroniza entre dispositivos ou sessões.
- Não há métricas públicas de uso, desempenho ou SEO. Metas de performance e acessibilidade estão documentadas internamente no repositório, mas não foram medidas e publicadas neste case.
- A suíte de testes end-to-end existe e está versionada, mas seus resultados não estão publicados neste documento.
- Uso em produção pela cliente e volume real de pedidos não são afirmados aqui.

Próximos passos reais

- Medir e publicar indicadores reais de performance (Core Web Vitals) e acessibilidade.
- Ampliar a cobertura de testes end-to-end conforme novas telas forem adicionadas.
- Capturar e publicar screenshots reais das telas, seguindo o plano de capturas do projeto.
- Avaliar, como melhoria não urgente, reduzir a exposição do endereço antigo da VPS ainda recuperável no histórico do repositório.

Como este projeto foi construído

O desenvolvimento foi conduzido por Wilian Gulini, com apoio de ferramentas de IA sob regras de projeto versionadas no próprio repositório (escopo, padrões de código, segurança e fluxos críticos documentados). Toda alteração passa por revisão humana e por validações automatizadas (lint, checagem de tipos e testes) antes de ser aceita.

Links públicos

- Repositório: `github.com/wiliangulini/floricultura-rosa-de-saron`
- Site: `rosa-de-saron.com`

O script de deploy versionado no repositório exige a variável `VPS_HOST` em vez de um host fixo. O endereço usado anteriormente ainda é recuperável no histórico do Git, sem que isso bloqueie a publicação deste link.

Fale comigo

Este projeto é parte do portfólio de Wilian Gulini, desenvolvedor front-end com foco em React, Next.js, TypeScript e Angular. Para ver outros projetos ou entrar em contato, acesse o portfólio ou o GitHub vinculados acima.